Práctica 4: Gestión de imágenes y contenedores

Materia:

Sistemas Distribuidos

Integrantes:

Elkin Yesid Yandun
Julian LeonardoCeron
Brayan Esmid Cruz
Jeison Enriquez

Docente:

LEIDY MARIANI DORADO FLÓREZ

Corporación Universitaria Comfacauca - Unicomfacauca

Popayán

2026

PARTE 1 – Exploración de Imágenes

Actividad 1: Ver imágenes locales

Al ejecutar el comando docker images, se listarán todas las imágenes de Docker que ya tienes en tu sistema.

```
docker images
```

```
PS C:\Users\brayan> docker images
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if th
e daemon is running: open //./pipe/dockerDesktopLinuxEngine: El sistema no puede encontrar el archivo especificado.
PS C:\Users\brayan> docker images
                                                                                                  i Info →   U  In Use
IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest     185d17a5489e        205MB         49.9MB
brayan:latest            57cf220b06a2        237MB         62.9MB
hello-world:latest       ef54e839ef54       25.9kB         9.52kB    U
mysql:latest             e5dc14f6e01c       1.27GB          283MB    U
nginx:latest             0d1b1f0656a7        240MB         65.8MB    U
```

- **¿Qué es el IMAGE ID?** Es un identificador único para cada imagen. Funciona como el “ADN” de la imagen, asegurando que no se confunda con ninguna otra.
- **¿Qué significa TAG?** Es una etiqueta que se le asigna a una imagen para identificar su versión. Por ejemplo, la imagen de mongo tiene los tags 7.0. Si no se especifica, por defecto se usa latest.
- **¿Qué representa SIZE?** Indica el espacio en disco que ocupa la imagen. Este tamaño incluye el sistema operativo base y todas las librerías y archivos que contiene.
- **¿Una imagen se está ejecutando?** No. Una imagen es una plantilla inerte, un paquete que contiene todo lo necesario para ejecutar una aplicación. No consume recursos de CPU o RAM por sí misma, solo espacio de almacenamiento.

Actividad 2: Descargar nuevas imágenes

Voy a descargar una nueva imagen desde Docker Hub, el registro público de imágenes. Usaré postgres como ejemplo con el comando.

```
docker pull postgres
```

```
PS C:\Users\brayan> docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
0480b5ae6db9: Download complete
967e70a4ad93: Pull complete
4e2113a73362: Download complete
eea32e6f70e8: Download complete
70f3de31936a: Pulling fs layer
e232269e9149: Pulling fs layer
be4ad6e904ed: Download complete
358db71ef364: Pulling fs layer
56a239f8dc77: Download complete
3859b312d253: Pulling fs layer
e7dee01ee448: Download complete
1b030cb6bea0: Pulling fs layer
90f0f713dfa1: Download complete
```

- **¿Qué cambió?** Apareció una nueva imagen en la lista postgres, junto con su IMAGE ID y SIZE.
- **¿El servicio ya está funcionando?** No. Al igual que antes, solo hemos descargado la plantilla. El servicio de postgres aún no está en ejecución.

PARTE 2 – Gestión de Contenedores

Actividad 3: Crear un contenedor

Usaré la imagen nginx que descargamos para crear y ejecutar un contenedor. Un contenedor es una instancia en ejecución de una imagen.

El comando:

```
Docker run -d --name servidor-web -p 8081:80 nginx
```

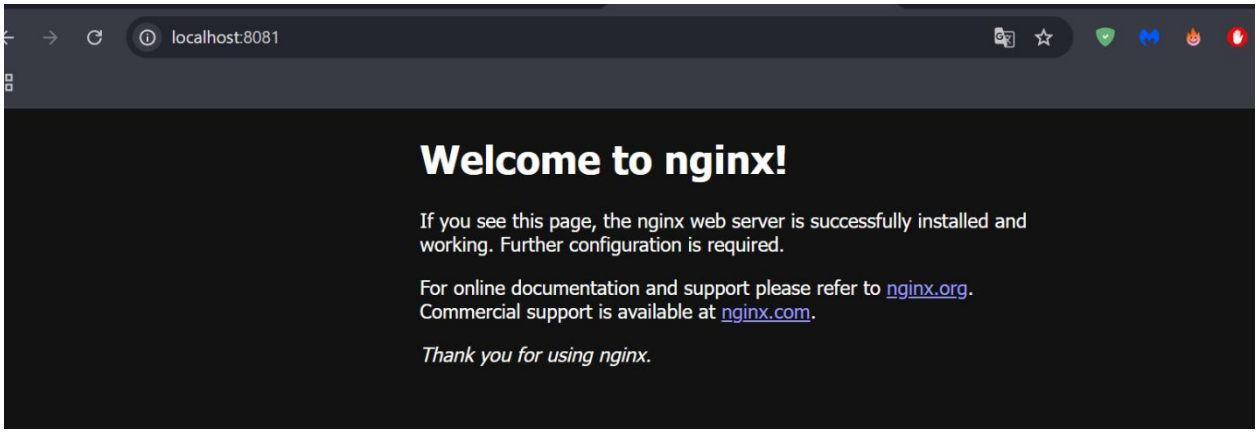
hace lo siguiente: “run” Crea y ejecuta un contenedor. “-d” Lo ejecuta en segundo plano. “--name servidor-web” Le asigna un nombre fácil de recordar. “-p 8081:80” Mapea el puerto 8081 de tu máquina al puerto 80 del contenedor, donde Nginx escucha. “nginx” Es la imagen que se utiliza como base.

Para verificar que está corriendo, usa.

```
docker ps
```

```
PS C:\Users\brayan> Docker run -d --name servidor-web -p 8081:80 nginx
225082a10622bccbc5860331529a1790977555029b701ec93d5d8d248ed99ad0
PS C:\Users\brayan> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS
     NAMES
225082a10622   nginx     "/docker-entrypoint.…"   13 seconds ago   Up 13 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/t
cp   servidor-web
```

Al abrir tu navegador e ir a http://localhost:8081, se verás la página de bienvenida de Nginx.



Actividad 4: Administrar el contenedor

Un contenedor tiene un ciclo de vida que se puede gestionar:

- **Detener:** “docker stop servidor-web” detiene la ejecución del contenedor. Si intento acceder a http://localhost:8081 ahora, no funcionará.

```
PS C:\Users\brayan> docker stop servidor-web
servidor-web
```

- **Ver todos:** “docker ps -a” muestra todos los contenedores, incluyendo los que están detenidos. Se mira que servidor-web tiene el estado “Exited”.

```
PS C:\Users\brayan> docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS
225082a10622   nginx         "/docker-entrypoint.…"   6 minutes ago   Exited (0) 2 minutes ago
fb6f70115cfe   nginx         "/docker-entrypoint.…"   2 days ago      Exited (0) 2 days ago
-1
fef8a25f183f   mysql         "docker-entrypoint.s…"   2 days ago      Exited (137) 2 days ago
1
7938676de72b   nginx         "/docker-entrypoint.…"   9 days ago      Exited (0) 9 days ago
n
db4e6815dd05   hello-world   "/hello"                 9 days ago      Exited (0) 9 days ago
ni
```

- **Reiniciar:** “docker start servidor-web” vuelve a iniciar el contenedor que habías detenido.

```
PS C:\Users\brayan> docker start servidor-web
servidor-web
```

- **Eliminar:** “docker rm -f servidor-web” elimina el contenedor de forma forzada (-f). Una vez eliminado, no se puede recuperar y se tendrá que crearlo de nuevo con “docker run”.

```
PS C:\Users\brayan> docker rm -f servidor-web
servidor-web
```

PARTE 3 – Análisis Conceptual

Aquí completamos la tabla comparativa basada en lo que hemos aprendido.

Concepto	Imagen	Contenedor
¿Es ejecutable?	No, es una plantilla.	Sí, es un proceso en ejecución.
¿Ocupa recursos en ejecución?	No (solo almacenamiento en disco).	Sí (CPU, RAM, red).
¿Puede modificarse?	No directamente, se crean nuevas versiones.	Sí, los cambios se guardan en su sistema de archivos.
¿Puede eliminarse?	Sí, con docker rmi.	Sí, con docker rm.

PARTE 4 – Reto en Equipo

Ejemplo usando mongo:

1. **Ejecutar contenedor:** “docker run -d --name mi-base-datos-mongo -p 27017:27017 mongo”

2. **Mostrar que está activo:** “docker ps”

```
PS C:\Users\brayan> docker run -d --name mi-base-datos-mongo -p 27017:27017 mongo
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
8d23bbf2b216: Pull complete
01d7766a2e4a: Pull complete
e46eb2190ffc: Pull complete
e79e499203c9: Pull complete
fbfd783f7a59: Pull complete
427c118b2079: Pull complete
cea2e6a72e6b: Pull complete
f97b55066391: Pull complete
4eeb70347fbd: Download complete
bae1425230be: Download complete
Digest: sha256:474f5c3bf0e355bb97dafda730e725169a4d51c5578abf7be9ec7ad3fdee4481
Status: Downloaded newer image for mongo:latest
8c86e79c43ce685a75d58be8c931077d16a3b44366dcf9c172ba2c0916300e40
PS C:\Users\brayan> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS
           NAMES
8c86e79c43ce   mongo     "docker-entrypoint.s…"   4 minutes ago   Up 4 minutes   0.0.0.0:27017
7017/tcp   mi-base-datos-mongo
PS C:\Users\brayan>
```

3. **Explicación:**

   - **¿Qué tipo de servicio es?** MongoDB es una base de datos NoSQL orientada a documentos. Almacena datos en formato BSON (similar a JSON), lo que la hace flexible y escalable para manejar grandes volúmenes de datos no estructurados o semiestructurados.

   - **¿En qué tipo de sistema distribuido podría usarse?** Es ideal para aplicaciones web modernas, sistemas de gestión de contenido, análisis de datos en tiempo real y microservicios que requieren una base de datos flexible y de alto rendimiento. Puede formar parte de un clúster distribuido para asegurar alta disponibilidad y escalabilidad horizontal.

   - **¿Qué pasaría si ese servicio falla?** Si el contenedor de MongoDB falla, la aplicación que depende de él para almacenar y recuperar datos dejará de funcionar correctamente. Esto podría resultar en la imposibilidad de guardar nueva información, acceder a datos existentes o incluso en la caída completa de la aplicación si no hay mecanismos de redundancia o recuperación implementados.

   -

PARTE 5 – Análisis Crítico

1. **¿Cuál es la diferencia real entre imagen y contenedor?** La comparacion más clara es la de clase y objeto en programación. La **imagen** es la clase (el plano o la plantilla), que define todo lo necesario. El **contenedor** es el objeto (la instancia de esa clase), que es la entidad viva y en ejecución que consume recursos.

2. **¿Por qué Docker facilita la escalabilidad?** Porque permite lanzar nuevas copias de un servicio (contenedores) en segundos con un solo comando. Si una aplicación web necesita más capacidad, simplemente se ejecutan más contenedores de Nginx o del backend. Este proceso, conocido como **escalado horizontal**, es rápido y eficiente.

3. **¿Cómo se relaciona esto con microservicios?** Docker es la tecnología ideal para microservicios. Cada microservicio (ej: servicio de pagos, de usuarios, de notificaciones) se empaqueta en su propia imagen y se ejecuta en contenedores aislados. Esto permite que cada servicio se desarrolle, actualice y escale de forma independiente sin afectar a los demás.

4. **¿Qué ventajas ofrece frente a instalar software directamente?**

   - **Portabilidad:** Un contenedor funciona igual en la laptop de un desarrollador, en un servidor de pruebas o en la nube.
   - **Aislamiento:** Los contenedores no interfieren entre sí. Puedes tener diferentes versiones de Python o Java corriendo en la misma máquina sin conflictos.
   - **Consistencia:** Elimina el problema de “en mi máquina sí funciona”, ya que la imagen empaqueta el entorno completo.

- **Rapidez:** Crear y destruir contenedores es mucho más rápido que configurar máquinas virtuales o instalar software manualmente.

Parte 6 - Conclusión

Conclusión grupal

Esta práctica nos ha permitido entender de primera mano la potencia de Docker para la gestión de aplicaciones. Hemos visto cómo las imágenes actúan como plantillas inmutables y los contenedores como instancias ejecutables y aisladas, lo que simplifica enormemente el ciclo de vida del software. La capacidad de Docker para empaquetar aplicaciones con todas sus dependencias en unidades portátiles es clave para la escalabilidad y la implementación de arquitecturas modernas como los microservicios. Sin duda, Docker es una herramienta fundamental que nos ayuda a evitar los clásicos problemas de compatibilidad y a desplegar nuestras aplicaciones de forma más eficiente y confiable en cualquier entorno